```
const express = require('express');
const fs = require('fs');
const path = require('path');
const createQLosu = require('./wasm/qlosu.m.js');
const app = express();
const PORT = process.env.PORT || 8080;
app.use(express.json({ limit: '4mb' }));
app.use(express.static(path.join(__dirname, 'www')));
app.use('/wasm', express.static(path.join(__dirname, 'wasm')));
let modulePromise = null;
let captureBuffer = '';
function getModule() {
  if (!modulePromise) {
    modulePromise = createQLosu({
      wasmBinary: fs.readFileSync(
        path.join(__dirname, 'wasm', 'qlosu.m.wasm')
      ),
      print: (t) => { captureBuffer += t + '\n'; },
      printErr: (t) => { captureBuffer += t + '\n'; },
    });
  }
  return modulePromise;
}
let queue = Promise.resolve();
function runCode(code) {
  const job = queue.then(async () => {
    const Module = await getModule();
    captureBuffer = '';
    const len = Module.lengthBytesUTF8(code) + 1;
    const ptr = Module._malloc(len);
    Module.stringToUTF8(code, ptr, len);
    const ret = Module._qlosu_run(ptr, len - 1);
    Module._free(ptr);
    return { stdout: captureBuffer, ret };
  });
  queue = job.catch(() => {});
  return job;
}
app.get('/api/version', async (req, res) => {
  try {
    const Module = await getModule();
    const ver = Module.UTF8ToString(Module._qlosu_version());
    res.json({ version: ver });
  } catch (e) {
    res.status(500).json({ error: String(e && e.message || e) });
  }
});
app.get('/api/examples', (req, res) => {
  const dir = path.join(__dirname, 'examples');
  const files = fs.readdirSync(dir)
```

```
set -euo pipefail
SCRIPT_DIR="$(cd "$(dirname "$0")" && pwd)"
ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"
OUT_DIR="${1:-$ROOT/wasm}"
LOSU_SRC="$ROOT/src/losu-quest/lpk-sdk/core"
APP_SRC="$ROOT/src/QLosu/Qapp/src/script/src"
QUEST_SRC="$ROOT/src/losu-quest/QuEST"
BUILD_TMP="$(mktemp -d /tmp/qlosu-wasm.XXXXXX)"
trap 'rm -rf "$BUILD_TMP"' EXIT
echo "==> 编译 Losu v2 内核 (emcc)"
mkdir -p "$BUILD_TMP/losu-obj"
for c in $(find "$LOSU_SRC" -name '*.c'); do
  emcc -O2 -I "$LOSU_SRC/include" -I "$APP_SRC" \
    -c "$c" \
    -o "$BUILD_TMP/losu-obj/$(basename "$c" .c).o"
done
echo "==> 编译 QuEST（WASM 适配版）"
mkdir -p "$BUILD_TMP/quest-build"
emcmake cmake -S "$QUEST_SRC" -B "$BUILD_TMP/quest-build" \
  -D ENABLE_MULTITHREADING=OFF -D MATH_LIBRARY="" \
  -D CMAKE_BUILD_TYPE=Release >/dev/null
grep -rl '"\-Ofast"' "$BUILD_TMP/quest-build" \
  | xargs -r sed -i 's/"-Ofast"/"-O3"/g' || true
emmake make -C "$BUILD_TMP/quest-build" \
  -j"$(nproc)" >/dev/null
echo "==> 生成 qufu.losu 内嵌模块"
python3 "$SCRIPT_DIR/gen_mod.py" "$APP_SRC/qufu.losu" \
  "$BUILD_TMP/qlosu_mod.c"
echo "==> 编译适配层"
emcc -O2 -I "$LOSU_SRC/include" -I "$APP_SRC" \
  -I "$QUEST_SRC" -I "$BUILD_TMP/quest-build" \
  -c "$SCRIPT_DIR/ffi_wasm.c" -o "$BUILD_TMP/ffi_wasm.o"
emcc -O2 -I "$LOSU_SRC/include" -I "$APP_SRC" \
  -I "$QUEST_SRC" -I "$BUILD_TMP/quest-build" \
  -c "$SCRIPT_DIR/qlosu_web.c" -o "$BUILD_TMP/qlosu_web.o"
emcc -O2 -I "$LOSU_SRC/include" -I "$APP_SRC" \
  -I "$QUEST_SRC" -I "$BUILD_TMP/quest-build" \
  -c "$SCRIPT_DIR/qlosu_ffi_reg.c" -o "$BUILD_TMP/qlosu_ffi_reg.o"
echo "==> 链接 WASM"
em++ -O2 "$BUILD_TMP/losu-obj/"*.o \
  "$BUILD_TMP/ffi_wasm.o" "$BUILD_TMP/qlosu_web.o" \
  "$BUILD_TMP/qlosu_ffi_reg.o" "$BUILD_TMP/qlosu_mod.o" \
  "$BUILD_TMP/quest-build/libQuEST.a" -o "$OUT_DIR/qlosu.m.js" \
  -s MODULARIZE=1 -s EXPORT_NAME=createQLosu \
  -s \
  EXPORTED_FUNCTIONS=_qlosu_version,_qlosu_run,_malloc,_free \
  -s \
  EXPORTED_RUNTIME_METHODS=stringToUTF8,lengthBytesUTF8,UTF8ToString \
  -s ALLOW_MEMORY_GROWTH=1 -s ENVIRONMENT=web,node --no-entry
echo "==> 完成：$OUT_DIR/qlosu.m.js / qlosu.m.wasm"
```

```
#ifdef NDEBUG
#undef NDEBUG
#endif
#include "losu_api.h"
#include "losu_base.h"
#include "losu_mem.h"
#include "losu_object.h"
#include "quest/include/quest.h"
#include <stdio.h>
static void
__release_QuESTEnv (void *vm, void *res)
{
  (void)(vm);
  (void)(res);
  losu_mem_free ((losu_vm_t)vm, res);
}
losu_base_ffi_declare (initQuESTEnv, vm, ctx, recall)
{
  (void)vm;
  (void)ctx;
  (void)recall;
  if (!isQuESTEnvInit ())
    initQuESTEnv ();
  QuESTEnv *env = losu_mem_malloc (vm, sizeof (QuESTEnv));
  *env = getQuESTEnv ();
  return losu_api_obj_context_return (
      vm, ctx,
      losu_api_obj_resource_new (vm, "C::QuEST::QuESTEnv", env, NULL,
                                 __release_QuESTEnv));
}
losu_base_ffi_declare (calcProbOfQubitOutcome, vm, ctx, recall)
{
  (void)vm;
  (void)ctx;
  (void)recall;
  struct losu_object_t arg1 = losu_api_obj_context_args (vm, ctx, 1);
  struct losu_object_t arg2 = losu_api_obj_context_args (vm, ctx, 2);
  struct losu_object_t arg3 = losu_api_obj_context_args (vm, ctx, 3);
  assert (losu_api_obj_type (vm, arg1) == losu_types_enum_t_resource
          && losu_api_obj_type (vm, arg2) == losu_types_enum_t_float
          && losu_api_obj_type (vm, arg3) == losu_types_enum_t_float);
  Qureg qureg = *(Qureg *)losu_api_obj_resource_get (vm, arg1);
  int bits = (int)losu_api_obj_float_get (vm, arg2);
  int outcome = (int)losu_api_obj_float_get (vm, arg3);
  qreal real = calcProbOfQubitOutcome (qureg, bits, outcome);
  return losu_api_obj_context_return_float (vm, ctx, real);
}
static void
__release_QuReg (void *vm, void *res)
{
```

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
<meta charset="UTF-8">
<meta name="viewport"
content="width=device-width, initial-scale=1.0">
<title>QLosu 量子编程平台 · 本地运行版</title>
</head>
<body>
<header>

<h1>❀ QLosu 量子编程平台</h1>

<span class="sub">
Losu v2 · Qufu 声明式框架 · QuEST 4.2.0 · WASM 本地运行
</span>
<span class="spacer"></span>
<span id="version">加载中…</span>
</header>
<main>
<aside id="example-list">
<h2>示例程序</h2>
</aside>
<section>
<div class="toolbar">
<span class="file" id="current-file">
untitled.losu
</span>
<span class="spacer"></span>
<button id="btn-run"
onclick="runCode()">
▶ 运行
</button>
</div>
<div id="editor-wrap">
<div id="lineno">1</div>
<textarea id="editor"
spellcheck="false"
oninput="syncLineno()"
onscroll="syncScroll()">
</textarea>
</div>
<div id="output-wrap">
<div id="output-head">
<span>输出</span>
<span id="status"></span>
</div>
<div id="output">
<span class="dim">
— 点击「运行」执行当前量子程序 —
</span>
```

```
  :root {
    --bg: #0b1020;
    --panel: #131a30;
    --panel2: #1a2340;
    --border: #2a3560;
    --text: #d8e0f5;
    --dim: #8a94b8;
    --accent: #4fc3f7;
    --quantum: #a78bfa;
    --ok: #4ade80;
    --err: #f87171;
  }
  * {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
  }
  body {
    background: var(--bg);
    color: var(--text);
    font-family: "Segoe UI", "PingFang SC", "Microsoft YaHei", sans-serif;
  height: 100vh;
  display: flex;
  flex-direction: column;
    overflow: hidden;
  }
  header {
  display: flex;
  align-items: center;
  gap: 14px;
    padding: 10px 20px;
    background: linear-gradient(90deg, #101a3a, #1a2340);
    border-bottom: 1px solid var(--border);
  }
  header h1 {
  font-size: 17px;
  font-weight: 600;
  letter-spacing: 1px;
    background: linear-gradient(90deg, var(--accent), var(--quantum));
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
  }
  header .sub {
    color: var(--dim);
    font-size: 12px;
  }
  header .spacer {
    flex: 1;
  }
  #version {
```